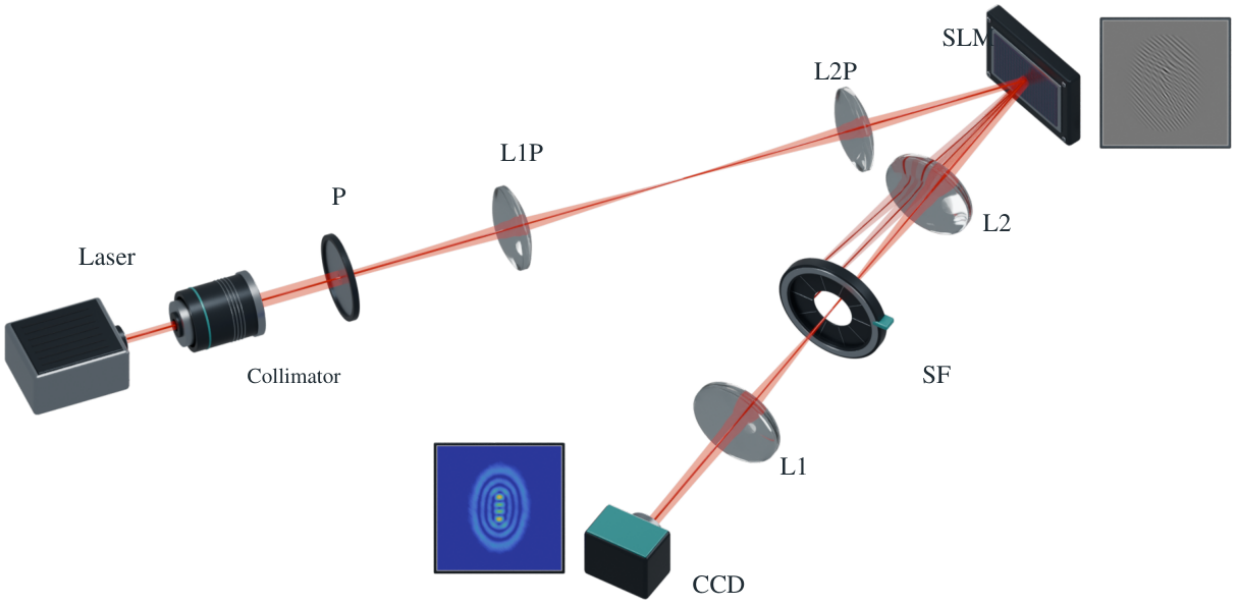
Blender Optical Schematics

From a fresh download to your first figure

B. M. Rodriguez-Lara

10 November 2026



What you will make. A Blender recreation of the optical setup illustrated in the paper, with an editable Blender file, a fixed view, a camera-search sampler and a selected view and SVGs with editable labels. This image is a reference preview from our later reconstruction, with the supplied SLM hologram and CCD intensity panels beside their instruments. Every step below uses this original paper-setup example. A bundled scene and tested workflow reproduce it without designing a second setup.

What inspired this project. The basic optical-setup schematic in https://doi.org/10.1103/6ftp-svh8 motivated the development of this Blender library. The paper contains no Blender model. The models and this example were created afterward; they were neither used in nor supplied with that publication. The paper DOI identifies the article, not this library.

No Blender coding experience is assumed. Follow the steps in order. The catalogue image is available separately at `images/catalogue-cover.png`.

1. Install the two applications

1. Install Blender from https://www.blender.org/download/. These assets were prepared with Blender 5.2.3 LTS. Use that version or a compatible newer version; older versions have not been tested.

2. Launch Blender once to finish its initial setup. Confirm that its normal scene window opens. You can close it afterward.

3. Install/open Codex from the official OpenAI application documentation, https://developers.openai.com/codex/app/, and sign in. Use a local project session that can read files and run Blender. A browser-only conversation without local file/execution access cannot perform this workflow on your computer.

Roles. Blender builds and renders the scene. Codex reads the agent instructions and writes/runs the build script. The supplied agent is a folder of instructions, not a third application or a Blender add-on. No API programming is needed for this walkthrough.

2. Download and keep the complete project together

1. If using GitHub, open this project's repository, choose Code, then Download ZIP. Extract the ZIP. Do not work inside the compressed ZIP. If you received a folder directly, use that complete folder.

2. Put the extracted project in a stable location, for example `Documents/OpticalElements/blender-optical-schematics/`. You may rename a downloaded folder ending in `-main` to `blender-optical-schematics`.

3. Open that folder in your file browser. You should see the following structure. This folder is called the **project root** in the remaining steps.

```
blender-optical-schematics/
  README.md
  library/                  Blender asset files
  agent/                    The agent's own folder
    README.md
    install.py
    setup-to-blender/
      SKILL.md
      agents/
      references/
  examples/paper-setup/      Scene, inputs and runnable workflow
  scripts/                  Camera, panel and beam helpers
  images/                   Catalogue and example preview
  docs/                     This PDF and its LaTeX source
```

Checkpoint. Keep `library/` and `agent/` here. Do not move them into Blender's application folder or split them across unrelated locations. All later relative paths start at this project root.

3. Register the library in Blender

These steps let you browse and drag components by hand. The agent's build script can also read the same library files directly.

1. Open Blender and open Preferences (normally Edit, then Preferences; on some macOS versions use the Blender application menu).
2. Find Asset Libraries. Depending on Blender version it is a separate preferences section or appears under File Paths.
3. Click the plus/add button. Browse to the project root and select its `library` folder. Select the folder, not an individual `.blend` file and not the `agent` folder.
4. Give it a recognizable name such as `Optical Schematics`. Save preferences if automatic saving is disabled.
5. In a Blender editor area's upper-left editor-type menu, choose Asset Browser. Select `Optical Schematics` in its library selector. If categories are empty, select All to see every asset.
6. Drag one collection into a fresh scene to confirm that it appears. You can discard this test scene. Do not save changes over a file inside `library/`.

Checkpoint. You can see optical components in the Asset Browser. Schematic assets use 1 Blender unit = 10 mm, with illustrative geometry. Catalogue thumbnails normalize sizes for legibility; use the source collection scale in your scene.

4. Open the correct folder in Codex

1. Use Codex's project/folder selection to open the `blender-optical-schematics` folder you saved in Step 2. Choose the project root, not `library/`, `agent/` or the parent folder containing both optical projects.
2. Start a chat attached to that local folder. Select local work if your interface offers a choice between local and remote execution.
3. Paste this readiness check. It does not ask Codex to generate the figure yet.

```
Check that this project contains library/, agent/, and
examples/paper-setup/. Locate Blender on this computer and
report its version. Do not generate a figure yet. If any
required file or application is missing, tell me exactly
what I need to fix before continuing.
```

Checkpoint. Codex confirms the three folders and finds Blender. If Blender is not on the system search path, give its executable location. On macOS it is commonly `/Applications/Blender.app/Contents/MacOS/Blender`. On Windows, locate `blender.exe` inside the installed Blender folder; the versioned folder name varies.

5. Put the agent in place

Recommended first run. Leave the complete `agent/` folder where it is. No extra installation is required when you call it by file path in Step 6. This avoids hidden-folder setup and automatic-discovery issues.

Optional named call. If you want to type `$setup-to-blender`, install a project-local copy. In Codex, paste:

```
Install this project's agent by running agent/install.py
with an available Python interpreter from this project
root. Do not install globally or overwrite a different
existing agent. Report the installed SKILL.md path.
```

The installer uses only standard Python. If Python is unavailable, the manual alternative is to copy the whole `agent/setup-to-blender` folder into `.agents/skills/` under this same project. Codex can perform that copy if you ask. The installed path must be:

```
blender-optical-schematics/.agents/skills/setup-to-blender/SKILL.md
```

The dot-prefixed `.agents` folder is normally hidden. Keep the visible `agent/` folder as the original distribution. After optional installation, start a new chat in the same project. If the named skill is still not recognized, use the direct call below; it does not depend on skill discovery.

6. Open the one example used throughout this guide

We keep the original setup: Laser, collimator, pinhole P, L1P, L2P, reflective SLM, L2, adjustable iris SF, L1 and CCD. The selected order passes SF; the rejected orders stop there. Physical focal lengths are unspecified. The pinhole is imaged onto the SLM with schematic spans $5, 10, 5$ BU (object–lens, lens–lens, lens–image), giving unit-magnitude inversion. The return relay uses the same spans, with SF in its common focal plane.

The asset audit confirms that P uses `SO_Pinhole`, SF uses `SO_Iris`, and the collimator uses `SO_Collimator`. The saved scene embeds these current assets.

The folder `examples/paper-setup/` now contains `source.blend`, `workflow.py`, `export.py`, the two original reference images, and `expected/`. The bundled scene is the exact starting point for every illustration below. The updated `layout.json` records optical-plane positions and the two validated 4F relays; instrument origins can differ from their active planes.

In Codex, the no-code route. Paste:

> Read examples/paper-setup/README.md. Use its bundled source.blend and workflow.py for the original paper setup. Reproduce the fixed view, coarse search, refined search and selected view, in order, in one new dated Figures folder. Run the ranking and paired SVG exports. Preserve the original optics, labels and data images. Show the sampler and explain the selected angle. Close task-opened application windows when finished.

Manual route used by the remaining commands.

1. Save any unrelated work open in Blender. The workflow opens the bundled scene.
2. Open Blender's Python Console using an editor's type selector. Paste the following, replacing only the project path and choosing an unused dated output folder.

```
import bpy, sys
from pathlib import Path
ROOT = Path('/path/to/blender-optical-schematics')
sys.path.insert(0, str(ROOT/'examples/paper-setup'))
import workflow
OUT = 'Figures/2026-11-10-Fig03'
```

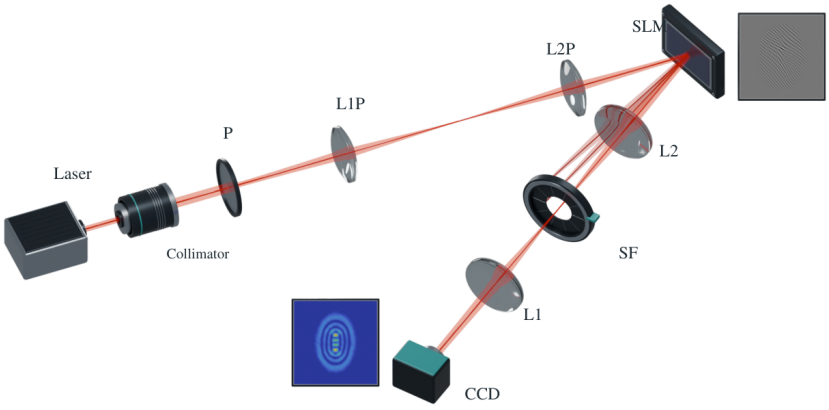
Use the same `OUT` in all later steps. Substitute your local date and an unused figure number. Calls that would replace a saved fixed or compact scene stop with an error; use the next version number within the same figure folder instead. The module `workflow` knows the project root, so this relative output path does not depend on Blender's current working directory.

7. Reproduce the fixed POV

In that same Blender console, run:

```
workflow.run('fixed', OUT)
```

This opens the original scene, reloads and packs its two bundled images, applies **Folded Relay — Front-Left**, re-anchors both image panels, saves `blender/fixed_v01.blend`, and renders `renders/fixed_v01.png`.



Fixed-view checkpoint. Your output should show these same ten labeled components and the same two data images. This is your fixed viewing direction applied to the original example. The camera position is $(-34, -20, 40)$ and target $(-3, -1, 1.6)$. Orthographic scale 44 fits this original setup; it changes framing without changing the viewing direction. Rendering uses 2400×1800 pixels.

The underlying function. The wrapper calls:

```
sys.path.insert(0, str(ROOT/'scripts'))
from camera_presets import apply_camera_preset
preset = ROOT/'library/camera-presets'
cam = apply_camera_preset(bpy.context.scene,
    preset/'folded-relay-front-left.json',
    ortho_scale=44, resolution=(2400, 1800))
workflow.update_reference_panels(bpy.context.scene, cam)
```

Use `target=` to reframe around another point, `ortho_scale=` to change the field of view, and `resolution=` to choose pixel dimensions. Distances are converted using the destination scene's units; `unit_scale=1` replays the stored coordinates literally. This example uses 0.01 metres per Blender unit, with illustrative rather than metric optical spacing.

8. Keep the same reference panels aligned

Changing a camera does not automatically orient or position its image panels. The workflow's callback reuses the original CCD and SLM panels. It keeps the CCD panel to the left of the CCD and the SLM panel to the right of the SLM, with the established lower-corner alignment and equal silhouette clearance.

Run after a manual camera change:

```
scene = bpy.context.scene
checks = workflow.update_reference_panels(scene, scene.camera)
```

The callback calls the following three functions from `panel_layout.py`. This is the CCD part of the actual example:

```
from panel_layout import (housing_corner,
    silhouette_edge, place_panel)
obj = bpy.data.objects['CCD']
anchor = housing_corner(scene, scene.camera, obj,
    'Camera housing', 'rear', 'left')
edge = silhouette_edge(scene, scene.camera, obj, 'left')
check = place_panel(scene, scene.camera,
    bpy.data.objects['CCD intensity'],
    bpy.data.objects['Reference frame'],
    anchor, 'left', .012, width=3.4, edge_u=edge)
```

`housing_corner`
: Finds the selected lower corner of the instrument housing in world coordinates.

`silhouette_edge`
: Finds the instrument's full projected edge, including protrusions, so the image panel clears the whole instrument.

`place_panel`
: Makes the image camera-parallel and aligns its lower inner corner. Here width 3.4 is in schematic Blender units and 0.012 is a fraction of the uncropped render width, not the final page border.

The SLM part uses `'SLM'`, `'SLM hologram'`, `'SLM reference frame'`, `'SLM housing'`, `'front'` and `'right'`. Use the supplied callback to update both together. Its output is saved as `fixed-panels.json` or `compact-panels.json`. Check that both panels face the camera, neither obscures an optic, and their lower corners align. The 0.5 mm page borders are applied later during SVG export.

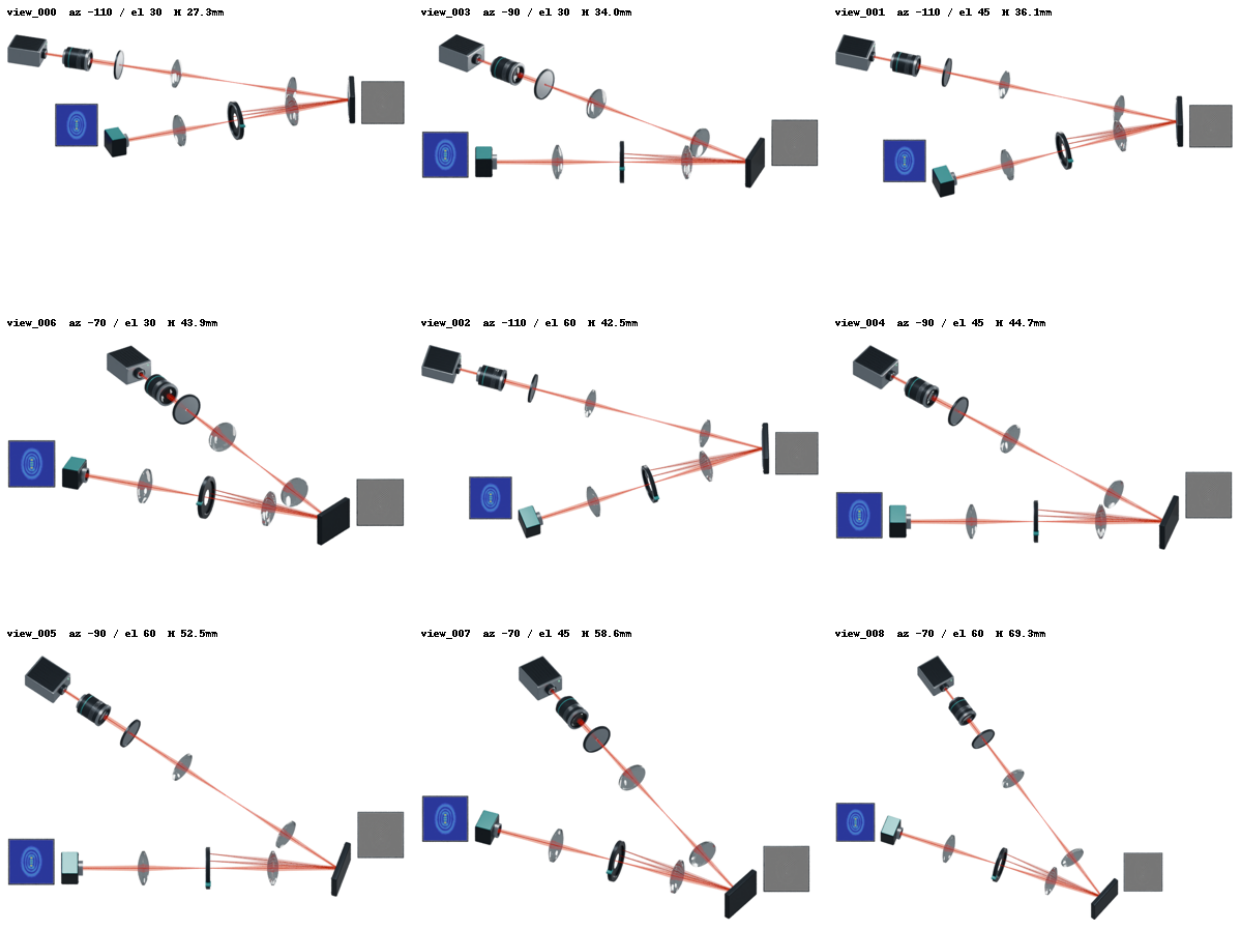
9. Render the coarse camera search

Keep the same optical scene and input images. In Blender, run:

```
workflow.run('coarse', OUT)
```

This produces nine previews in `OUT/coarse/`: three azimuths $(-110, -90, -70)^\circ$ and three elevations $(30, 45, 60)^\circ$. Each PNG has a camera JSON; `candidates.json` records the list. The original camera and panel transforms are restored when the sweep finishes.

Actual sampler of this same original example



How the wrapper calls the search function:

```
from search_camera_views import render_camera_candidates
render_camera_candidates(bpy.context.scene,
    ROOT/OUT/'coarse', target=(-3, 0, 1.6),
    azimuths=[-110, -90, -70], elevations=[30, 45, 60],
    ortho_scale=44, distance=60,
    resolution=(1000, 800), samples=16,
    update_panels=workflow.update_reference_panels)
```

This code explains the call already performed above; do not run both into the same directory. The output directory must be new. Azimuth runs from world $+X$ toward $+Y$; elevation is above the XY plane. The target, distance and orthographic scale are Blender units. Keep these framing settings consistent across candidates. The callback is required here because the example has image panels.

10. Rank the coarse views and refine the search

Outside Blender, open a terminal at the project root. Use a Python interpreter with Pillow. Replace the output path with the same folder chosen in Step 6:

```
python scripts/rank_camera_views.py Figures/2026-11-10-Fig03/coarse
```

Open `coarse/comparison.png` and `coarse/ranked.json`. The script calls `rank(directory)`; its `measure(image)` function reports crop bounds, cropped page height, occupied fraction and approximate empty page area at 86 mm width. Empty or clipped previews are flagged invalid. Transparent glass and beams make coverage threshold-dependent; the ranking supports visual judgement rather than replacing it.

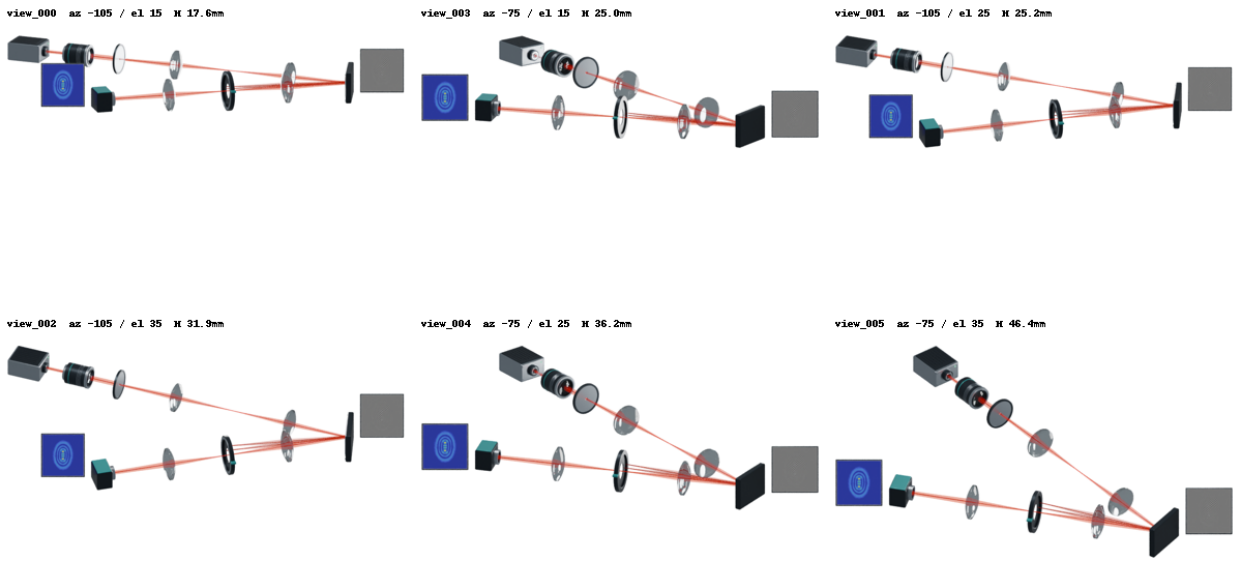
Refine around promising directions. In Blender:

```
workflow.run('refine', OUT)
```

Then in the terminal:

```
python scripts/rank_camera_views.py Figures/2026-11-10-Fig03/refine
```

This tests azimuths $(-105, -75)°$ and elevations $(15, 25, 35)°$. The panel callback and optical geometry are unchanged.



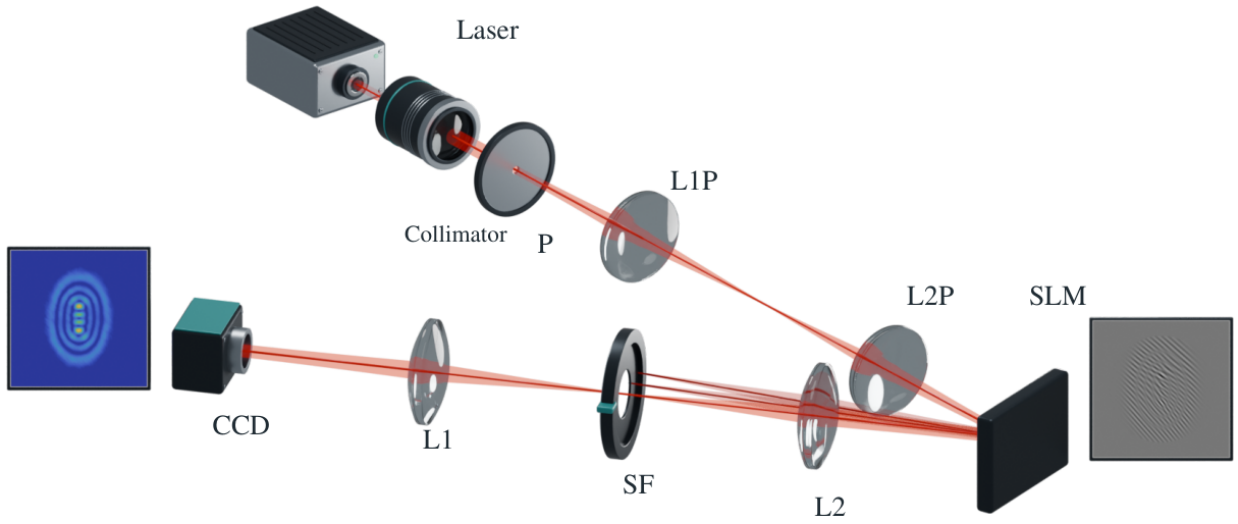
Checkpoint. Compare both samplers. Check the SLM face, pinhole and iris, all four lenses, separation of the beam branches, image panels and room for labels. A numerically smaller view may make an aperture edge-on or hide a lens. Inspect before choosing. The bundled samplers are under `examples/paper-setup/expected/`; they are previews of this original example, not of a different relay.

11. Apply and save the selected POV

For this walkthrough, use the included compact oblique direction, $-75°$ azimuth and $25°$ elevation. The original setup remains fixed. Run in Blender:

```
workflow.run('compact', OUT)
```

This saves `blender/compact_v03.blend`, versioned camera, panel and anchor JSON files in `json/`, and `renders/compact_v03.png` next to the fixed-view results. It applies the named preset with scale 44 and a 2400×1800 canvas, then re-anchors the panels and renders at 64 samples.



Selecting another tested candidate. Each sampler PNG has a camera JSON. For example, to inspect the middle coarse view in the current Blender scene:

```
cam = apply_camera_preset(bpy.context.scene,
    ROOT/OUT/'coarse/view_004.json',
    resolution=(2400, 1800))
workflow.update_reference_panels(bpy.context.scene, cam)
```

Read that JSON's azimuth and elevation before choosing it. Save a separate Blender variant and regenerate label positions for the new camera. Do not overwrite the supplied presets. The exporter in the next step includes checked label layouts for `fixed` and `compact`; another angle needs its own lettering layout.

What remains invariant. Component positions, identities, optical topology and data images are the same in both figures. This is a comparison of viewing direction and framing. It is not a change of focal lengths, a diffraction calculation or proof of a globally optimal camera.

Readability check. This alternative shows the back of the SLM. Keep the fixed view when the active face must be visible. This is a demonstration of applying a candidate, not a claim of an optimal view.

11a. Restore the beams and illuminate the chosen view

Choose and save the camera first. Keep that camera and the component positions fixed while finishing the appearance. The preceding samplers compare composition with the restored translucent beams; finish the lighting after choosing the camera.

Apply the finishing step. In the same Blender console, after Step 11:

```
workflow.finish('compact', OUT, samples=128,
                version=4, source_version=3)
```

This opens `blender/compact_v03.blend`, preserves its approved beam materials, and places three area lights relative to the selected camera. It saves `blender/compact_v04.blend`, a versioned lighting report in `json/`, and `renders/compact_v04.png`. The camera and component transforms are checked for equality. Images are packed into the scene. Run `workflow.finish('fixed', OUT)` to illuminate the fixed view too. For another revision, use `version=5, source_version=4`.

Position lights in the camera frame. Camera-right is $+x$, camera-up is $+y$, and toward the camera is $+z$. Each light points at the setup target $(-3, 0, 1.6)$. These are schematic Blender units, not physical experiment dimensions.

Light	x	y	z	Power	Size
Key	-12	14	18	3300 W	11
Fill	14	5	12	1100 W	12
Rim	4	12	-10	2200 W	8

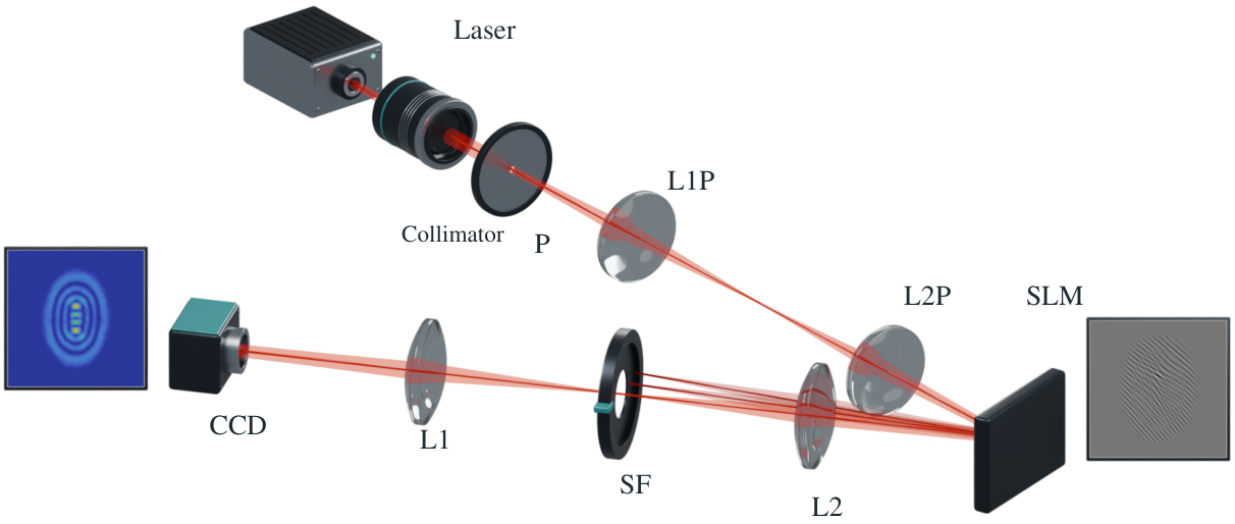
The large upper-left key creates different brightness on adjacent housing faces. The weaker right fill keeps dark faces readable without flattening them. The rear rim supplies lens-edge and housing-edge highlights. World strength 0.25 supplies restrained ambient illumination. Existing scene lights are muted to avoid doubling illumination. The three named rig lights are reused when called again.

Tune and preview. For adjustments on the currently open finished scene, call the helper directly; do not rerun the workflow wrapper over an existing finished file.

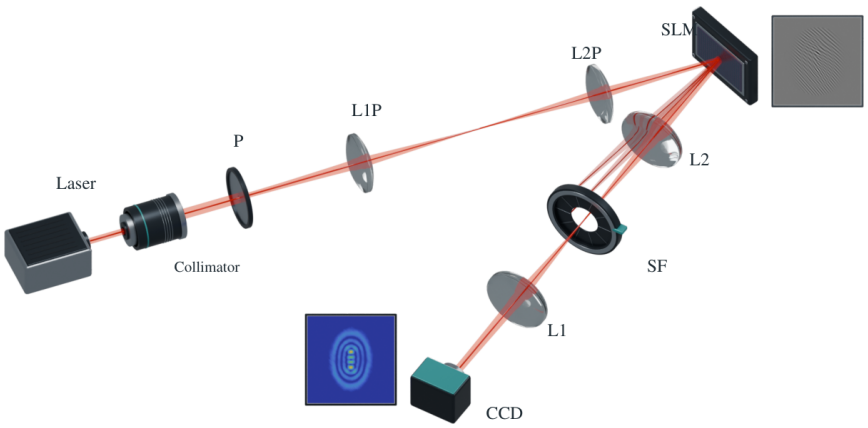
```
from finish_scene import finish_scene
scene = bpy.context.scene
finish_scene(scene, target=(-3,0,1.6), restore_beams=False,
    key_power=3300, fill_power=1100,
    rim_power=2200, world_strength=.25)
scene.cycles.samples = 16
scene.render.filepath = str(ROOT/OUT/'lighting-preview.png')
bpy.ops.render.render(write_still=True)
```

If the image looks flat, reduce fill or ambient strength before increasing the key. If dark housing faces disappear, raise the fill modestly. If lens highlights obscure the beam, lower the rim power or move its light sideways. For a position adjustment, edit the camera-frame offsets in `scripts/finish_scene.py`; rerun the helper. If the camera changes, rerun the helper and the panel-placement callback.

11b. Inspect and save the final appearance



Finished compact view of the same original example. Lens highlights, shaded housings and translucent red envelopes restore depth. The core remains readable on white. The SLM's back is still visible in this direction; lighting does not correct a poor viewing direction.



The same finishing recipe on the fixed camera. Light positions follow this camera's axes. The active SLM face is visible here.

Beam settings. Selected and rejected envelopes share red $(1, .008, .003)$, with emission strengths 0.7 and 0.336 and transparent/emission mix factors 0.21 and 0.14. Thin shaded cores use emission strength 0.2. Their radius is capped at 80 percent of the narrowest local envelope radius. Rejected orders are dimmer red. These are illustrative envelopes, not simulated diffraction or measurements.

Final checks. View the transparent PNG on white. Confirm separate beam branches, visible apertures, gentle lens highlights, shaded housing faces and readable image panels. Use AgX, denoising and 128 samples for the new illuminated version; increase samples if a close inspection shows noise. After manual tuning, render to `renders/compact_v04.png` and save `blender/compact_v04.blend`. Preserve an approved version by saving later changes with the next suffix. Then export the finished render in Step 12.

12. Export and inspect both SVG pairs

Run these commands in a terminal at the project root, outside Blender:

```
python examples/paper-setup/export.py \
  Figures/2026-11-10-Fig03 fixed --version 4
python examples/paper-setup/export.py \
  Figures/2026-11-10-Fig03 compact --version 4
```

The function `export(output, stage, width_mm=86, border_mm=.5)` crops the transparent PNG to its content, preserves its aspect ratio and places it 0.5 mm from every page edge. It produces:

```
Figures/2026-11-10-Fig03/
  python/                    Figure-specific scripts
  blender/                   fixed_v01, compact_v03, compact_v04
  json/                      Versioned camera/lighting/checks
  md/                        Notes and approval status
  txt/                       Execution logs
  source/                    Input data images
  renders/                   compact_v04.png and previews
  svg/                       compact_lettered_v04.svg
                             compact_unlettered_v04.svg
  coarse/                    Nine camera candidates
  refine/                    Six camera candidates
  FinalFigure/               Empty until visual approval
```

1. Open the lettered SVG in a vector editor. Check that every label names the correct component and that none collides with an optic or panel.
2. Confirm the document width is 86 mm and all four outer borders are 0.5 mm. The height follows the cropped render; it is not a fixed paper size.
3. Edit text in the `lettering` layer. The `render` layer is an embedded raster image. The unlettered partner uses the same render and dimensions.
4. Check the JSON reports and obtain visual approval before marking the figure final. Close application windows you opened after saving your work.

Use `-width-mm 129` or `-width-mm 172` for wider publication columns. The border remains 0.5 mm unless explicitly overridden with `-border-mm`. Lettering positions in the supplied exporter are calibrated to this one example and its two documented camera views.

12a. Approve and collect the final version

One folder per figure. Keep the date and figure number fixed across revisions. Use consecutive `FigXX` numbers within each date and project, with exact capitalization. Append `_vXX` to filenames, before the extension. A new camera or lighting revision does not need another figure folder. Reusable scripts and libraries retain their project locations.

Composition is not final approval. The selected POV may still look flat. Finish the lighting in Steps 11a–11b and inspect the same original example on white. Check shaded housing faces, clear iris and pinhole openings, restrained highlights, approved beam appearance and unobstructed reference panels. If appearance is not accepted, keep it as a working version and leave `FinalFigure/` empty. Keep an already approved set until its replacement is approved.

After approval. For the illuminated compact view from Step 12, run:

```
python scripts/finalize_figure.py \
  Figures/2026-11-10-Fig03 \
  --stage compact --version 4 --approved
```

The `-approved` flag records approval already given by the figure owner; it is not an automatic aesthetic test. The helper requires the scene and both SVGs, checks matching SVG dimensions, copies them, and verifies file hashes. It refuses to overwrite different approved files with the same names. The result is:

```
FinalFigure/
  compact_v04.blend
  compact_lettered_v04.svg
  compact_unlettered_v04.svg
json/
  compact-approval_v04.json
```

Earlier versions remain in their type folders. Close task-opened Blender and viewer windows after saving. The complete convention is also in `docs/figure-workflow.md`.

Panel clearance and schematic hardware. The original example keeps its existing panels. When adding a translation rail in another figure, use the library's minimal rail and carriage, with the CCD directly on the carriage. If the lower-left anchor creates overlap, call `housing_corner(..., vertical='top')` for the CCD rear left corner. Align the panel's lower-right corner vertically to it, measure the horizontal gap from the entire instrument silhouette, and check projected overlap with every component. Recompute the placement after camera changes.

13. Use the assets and beam functions on the same example

The bundled scene uses collection instances from the schematic library. Select an instrument instance in Blender to inspect its `instance_collection`; do not remodel it or save changes over the source library. The four lenses reuse `SO_Biconvex`. Their illustrative shape does not assign a focal length.

Reference-image replacement. To try your own data while keeping this example, copy `examples/paper-setup/` to a separate project copy, replace `reference-images/slm_hologram.png` and `ccd_intensity.png`, and run the workflow into a new output folder. `load_example()` reloads these files and packs them; `update_reference_panels()` updates alignment. Do not invent missing data images.

Beam representations. The original scene retains its Gaussian-volume geometry. The camera-search previews use the restored translucent envelopes and shaded cores. Steps 11a–11b preserve that approved beam appearance while adjusting illumination. Explicit restoration is available with `restore_beams=True`. Paths and endpoints are unchanged. Both representations are illustrations rather than simulations.

The functions used for these two representations are:

```
from beam_envelope import create_envelope
from gaussian_beam import create_gaussian_segment
```

To experiment in a separate scene copy, select one existing beam segment, record its endpoints and radii, hide that segment, and construct its replacement in a dedicated collection. For the original example's L1P-to-waist segment:

```
c = bpy.data.collections.new('Beam style trial')
bpy.context.scene.collection.children.link(c)
a, b = (-4, 5, 1.6), (0, 2, 1.6)
beam = create_envelope('Input relay trial',
    a, b, .18, .018, c)
```

`create_envelope` takes 3D endpoints, start/end radii and a collection. The alternative `create_gaussian_segment` takes XY endpoints with `optical_height=1.6`, the same radii and collection. Use one representation per segment, not overlapping replacements. Set `rejected=True` for a rejected order. Rejected paths remain red and dimmer; they stop at SF. Rebuild both views and their exports after any beam change.

This optional style exercise still uses the same optical setup. Translation accessories and the experimental laptop are catalogued elsewhere; they are not inserted as a second example in this guide.

14. Function map and troubleshooting

`workflow.load_example()`
: Opens the bundled original scene and packs the original reference images. Save unrelated open work first.

`workflow.run(stage, output)`
: Runs `fixed`, `coarse`, `refine` or `compact`. Relative output paths start at the project root. Creates figure outputs without replacing prior saved variants.

`apply_camera_preset(scene, path, ...)`
: Restores pose and framing from JSON; optional target, unit scale, orthographic scale and resolution are described in Step 7.

`workflow.update_reference_panels(scene, camera)`
: Calls `housing_corner`, `silhouette_edge` and `place_panel` for both original panels; see Step 8.

`render_camera_candidates(scene, directory, ...)`
: Renders an angle grid with the panel callback, saves each camera JSON and restores the scene; see Step 9.

`measure(image)` and `rank(directory)`
: Measure alpha bounds/coverage and publication footprint, flag invalid previews, and build a ranked comparison; see Step 10. These use ordinary Python with Pillow, not Blender Python.

`finish_scene(scene, ...)`
: Positions camera-relative studio lights; optionally restores beams; see Steps 11a–11b.

`finalize(folder, stage, version, approved)`
: Copies the approved scene and SVG pair with hash verification; see Step 12a.

`export(output, stage, ...)`
: Writes the fixed or compact SVG pair and dimension report; see Step 12.

`create_envelope` / `create_gaussian_segment`
: Create illustrative beam geometry in a collection; see Step 13.

Missing import. Ensure the console bootstrap in Step 6 used your real project root. Scripts run inside Blender need Blender's Python; ranking and export need external Python and Pillow.

Existing output error. Choose a fresh dated folder or a new sweep subdirectory. Do not delete the accepted result to rerun a comparison.

Clipped preview or missing panel. Check the supplied images and panel callback. Increase the common orthographic scale or canvas dimensions for the whole sweep, then rerun it consistently. Regenerate lettering if framing changes.

Background Blender fails. Run the same `workflow.run` commands in Blender's Python Console. The functions are identical; a GUI fallback does not require rebuilding the optics. Save results and close the task window after completion.

Unexpected scientific interpretation. Read `examples/paper-setup/README.md`. P is a pinhole, L2 is nearest SLM, L1 is nearest CCD, and focal lengths are unknown. Beam widths and mounting geometry are illustrative. The paper supplied the inspiration, not these Blender models.

15. Citation, rights and editable guide

When using this library to prepare a figure, cite the project using its stable DOI:

Rodriguez-Lara, B. M. (2026). *Blender Optical Schematics* [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.23993947

Stable project DOI. This DOI identifies Blender Optical Schematics across all releases and opens the latest archived version. It remains the same as the library develops.

BibTeX for LaTeX

Copy this entry into your bibliography `.bib` file, or use the supplied `docs/citations/schematics-project.bib`. The `@misc` entry works with traditional BibTeX styles; the note identifies the work as computer software.

```
@misc{rodriguez_lara_2026_optical_schematics,
  author    = {Rodriguez-Lara, B. M.},
  title     = {{Blender Optical Schematics}},
  year      = {2026},
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.23993947},
  url       = {https://doi.org/10.5281/zenodo.23993947},
  note      = {Computer software}
}
```

In your manuscript, use `\cite{rodriguez_lara_2026_optical_schematics}` and compile with your normal bibliography workflow.

Other formats and reference managers

- **RIS:** import `docs/citations/schematics-project.ris` into a reference manager such as Zotero or EndNote.
- **CSL-JSON:** use `docs/citations/schematics-project.csl.json` with tools supporting Citation Style Language data.
- **Plain text:** copy `docs/citations/schematics-project.txt` and adapt the formatting to your journal's style.

Cite the library for the Blender assets. Use the library DOI above when using these models. Cite the paper at https://doi.org/10.1103/6ftp-svh8 only when referring to its optical setup or scientific results. Its basic schematic inspired this project; the article contains no Blender model. The models were created afterward and were neither used in nor supplied with the publication. Library citation does not replace the attribution required by CC BY 4.0 for reused models, figures and documentation.

The editable source is `docs/user-guide.tex`. Compile twice with `pdflatex user-guide.tex` from `docs/`. Original code uses MIT; authored models and documentation use CC BY 4.0, subject to the exclusions in LICENSE and `docs/provenance.md`. The two original scientific reference images retain their separately documented rights.